



DevOps

Introduction

Structure



Cours -> Exam
(Quiz on CV)



Activity -> Project
(practical or 6 pages IEEE paper)



Final Grade =
 $(\text{exam grade} + \text{activity grade}) / 2$

Main topics

Theoretical competences

- What is DevOps and why should we care about it
- Cloud Native
- Microservices architecture and ways to implement it

Practical competences

- CI/CD with Azure DevOps
- Azure PaaS for microservices
- Infrastructure as Code (Terraform)

+

•

○

Bibliography

[DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations](#)

by [Gene Kim](#) (Author), [Jez Humble](#) (Author), [Patrick Debois](#) (Author), [John Willis](#) (Author), [Nicole Forsgren](#) (Author)

+

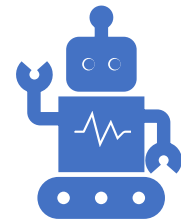
•

○

Other Bibliography

- Phoenix Project: A Novel About It, Devops, And Helping Your Business Win
- The Unicorn Project: A Novel about Digital Disruption, Redshirts, and Overthrowing the Ancient Powerful Order: A Novel about Developers, Digital Disruption, and Thriving in the Age of Data

DevOps: The secret sauce
behind every successful tech
company.





Menti Question

Why DevOps is important?

Why DevOps is important?

- Enable **fast flow** of planned work into production while ensuring **stability, reliability, availability, and security**
 - Identify which feature will have highest impact on customers and advanced organization goals
 - New feature smoothly and frequently without disruption (chaos)
 - Reduce friction to increase productivity and have better outcomes
 - Small teams can develop and deploy quickly, independently, securely, reliably





Menti Question

What problems can occur without DevOps?

Life without DevOps

- Develop feature that don't bring business value
- Critical activity **take forever**
- Too many **handoffs**
- **Poor quality**
- **Negative impact** on customer and business
- **Unhappy employees**



DevOps (software) vs Lean (manufacturing)



Before Lean

Order lead time 6 weeks
< 70% of order on time



After Lean

Order lead time 3 weeks
> 95% of order on time

Evolution of feature development

- Source: Adrian Cockcroft, “Velocity and Volume (or Speed Wins),” presentation at FlowCon, San Francisco, CA, November 2013.
- Kim, Gene; Humble, Jez; Debois, Patrick; Willis, John; Forsgren, Nicole. The DevOps Handbook (p. 36). IT Revolution Press. Kindle Edition.

	1970s–1980s	1990s	2000s–Present
Era	Mainframes	Client/Server	Commoditization and Cloud
Representative technology of era	COBOL, DB2 on MVS, etc.	C++, Oracle, Solaris, etc.	Java, MySQL, Red Hat, Ruby on Rails, PHP, etc.
Cycle time	1–5 years	3–12 months	2–12 weeks
Cost	\$1M–\$100M	\$100k–\$10M	\$10k–\$1M
At risk	The whole company	A product line or division	A product feature
Cost of failure	Bankruptcy, sell the company, massive layoffs	Revenue miss, CIO's job	Negligible

Menti Question: What can we learn?

What is the current context?



“Every industry and company that is not bringing software to the core of their business will be disrupted.” (Jeffrey Immelt, CEO of General Electric)

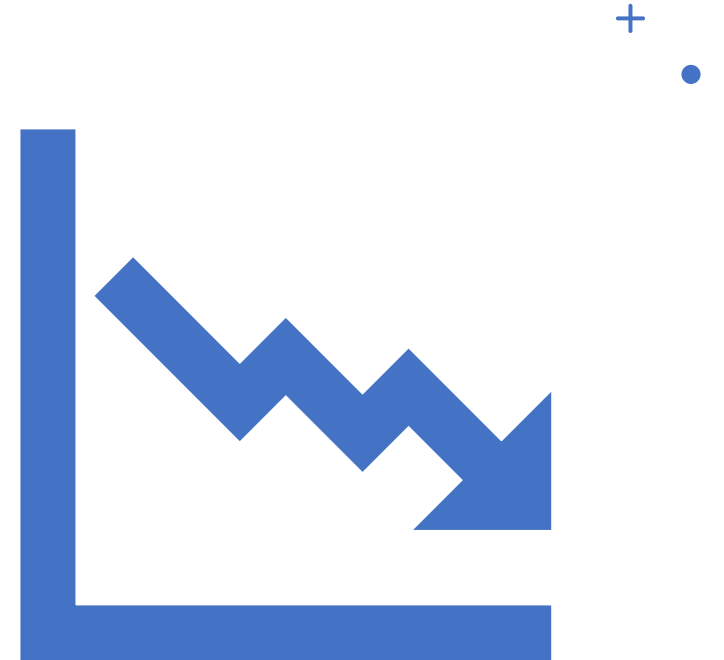


“In previous economic eras, businesses created value by moving atoms. Now they create value by moving bits.” (Jeffrey Snover, Technical Fellow at Microsoft)

Development and IT operations (without DevOps)

=> conflict

- Slow time to market
- Reduced quality
- Increased outages
- Increasing technical debt

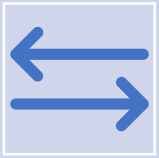




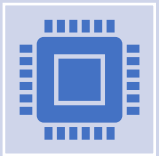
Menti Question

Why there is a conflict between
Development and IT Operations?

Conflicting goals



Respond to the rapidly changing competitive landscape
(Development)



Provide stable, reliable, and secure service to the customer
(IT Operations)

Kim, Gene; Humble, Jez; Debois, Patrick; Willis, John; Forsgren, Nicole. The DevOps Handbook (p. 38). IT Revolution Press.
Kindle Edition.

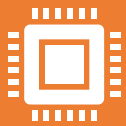


Technical debt

decisions we make lead to problems that get increasingly more difficult to fix over time,
continually reducing our available options in the future

Kim, Gene; Humble, Jez; Debois, Patrick; Willis, John; Forsgren, Nicole. The DevOps Handbook (p. 38). IT Revolution Press. Kindle Edition.

Downward Spiral in Three Acts



technical debt and daily workarounds => IT Operations keeps applications and infrastructure running despite many problems due to complex applications and infrastructure that poorly documented, and incredibly fragile



Compensate for a broken promise made by product manager to deliver unrealistic goals => development team has another urgent and technically challenging project, hence they will cut corners



Everything gets a little bit more difficult => smaller changes causes bigger failure



Menti Question

How can we break the Downward Spiral?

Breaking the Downward Spiral with DevOps



Small teams of developers independently implement their features



Validate their correctness in production-like environments



Have their code deployed into production quickly, safely, and securely



Code deployments are routine and predictable



Deployments occur throughout the business day



Customers don't notice—except when they see new features and bug fixes that delight them



Main idea behind DevOps
Short feedback loop



Menti Question

How can we collect feedback in SW development?

Automated testing



are run in production-like environments



give continual assurance that the code and environments operate as designed and are always in a secure and deployable state



helps developers discover their mistakes quickly (minutes)



enables faster fixes as well as genuine learning



NO technical debt, problems are fixed as they are found

Telemetry



In code and production environments ensures that problems are detected and corrected quickly



Confirms that everything is working as intended



Confirms that customers are getting value from the software they use

DevOps Impact on teams

Small teams work safely and architecturally decoupled from the work of other teams

Use self-service platforms that leverage the collective experience of Operations and Information Security

Even high-profile product and feature releases become routine by using dark launch techniques

New functionality release through a feature toggle or configuration setting

Features are released gradually

Releases are predictable, reversible, and low stress

DevOps impact on teams

Encourage learning

Reduces the risk of working long periods on features that are not needed by customers

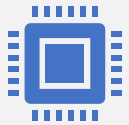
High-trust and collaborative culture

Everyone fully owns the quality of their work

Blameless post-mortems

Design for failure

The business value of DevOps



Throughput metrics

code and change deployments (**30x** more frequent)

code and change deployment lead time (**200x** faster)



reliability metrics

production deployments (**60x** higher change success rate)

mean time to restore service (**168x** faster)

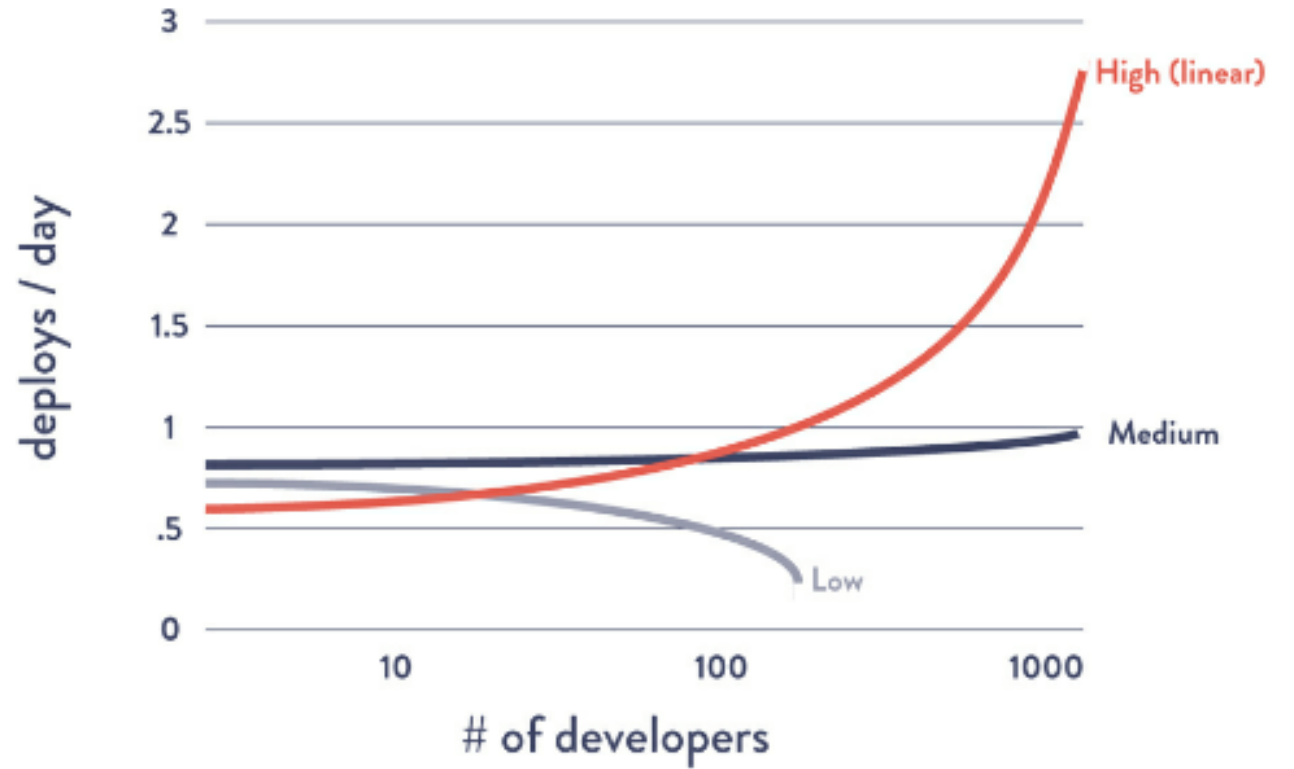


Organizational performance metrics

productivity, market share, and profitability goals (**2x** more likely to exceed)

market capitalization growth (**50%** higher over three years)

DevOps enable scaling development



Puppet Labs, 2015 State Of DevOps Report.

DevOps influenced by

Lean (manufacturing) - Toyota

Small batch sizes of work => short lead times

Scientific thinking

Pull vs push

Quality at the source

Respecting every individual

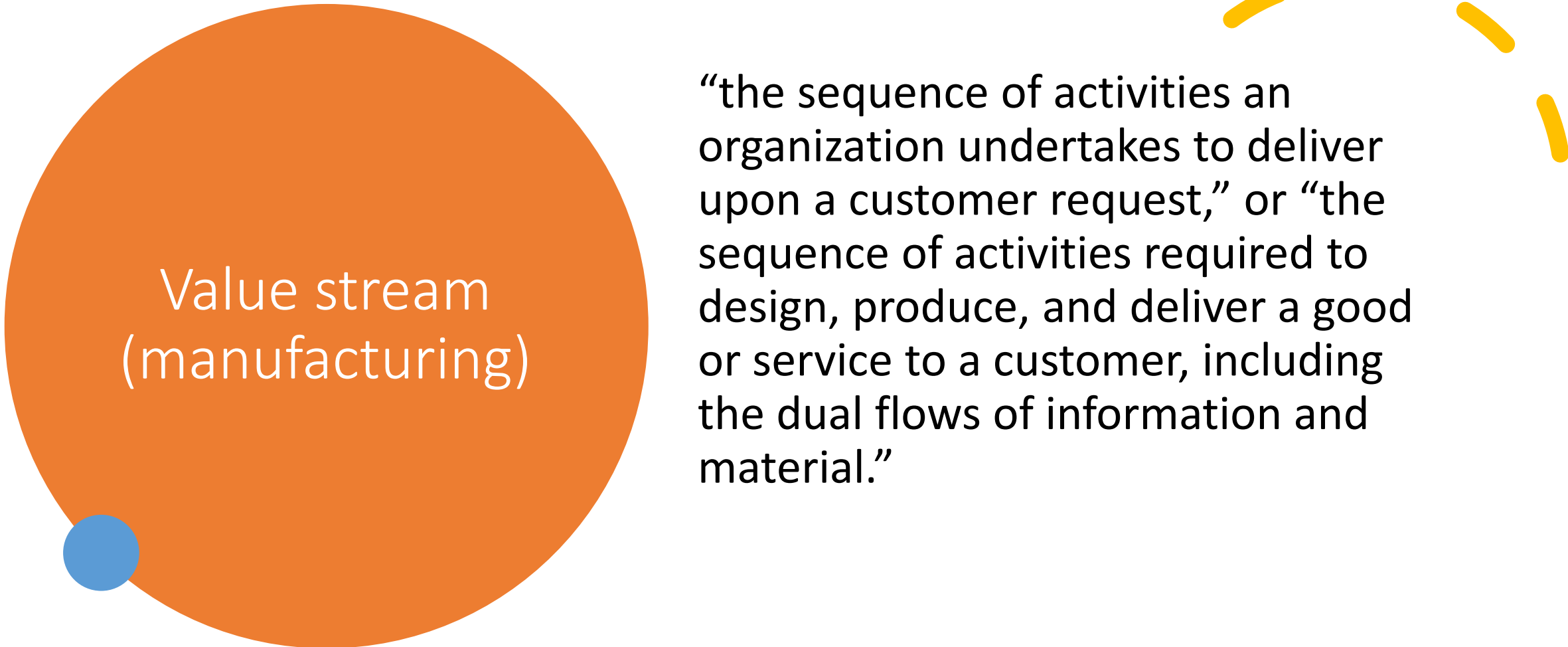
Agile Manifesto

Deliver working software frequently

Short batches

Incremental release

Small, self-motivated teams



Value stream (manufacturing)

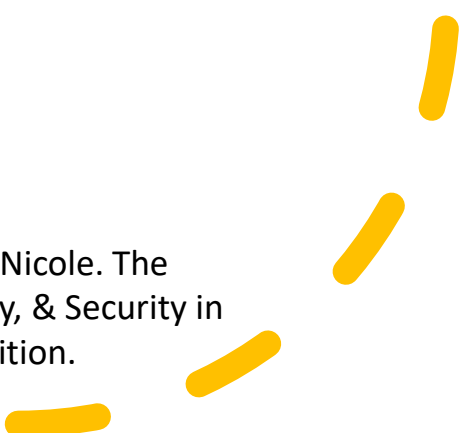
“the sequence of activities an organization undertakes to deliver upon a customer request,” or “the sequence of activities required to design, produce, and deliver a good or service to a customer, including the dual flows of information and material.”



Value stream (technology)

“the process required to convert a business hypothesis into a technology-enabled service or feature that delivers value to the customer”

Kim, Gene; Humble, Jez; Debois, Patrick; Willis, John; Forsgren, Nicole. The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations (p. 64). IT Revolution Press. Kindle Edition.



Deployment lead time

This value stream begins when any engineer in our value stream (which includes Development, QA, IT Operations, and Infosec) checks a change into version control and ends when that change is successfully running in production, providing value to the customer and generating useful feedback and telemetry.

New feature development has 2 phases



Design and Development (Lean Product Development) – high variability



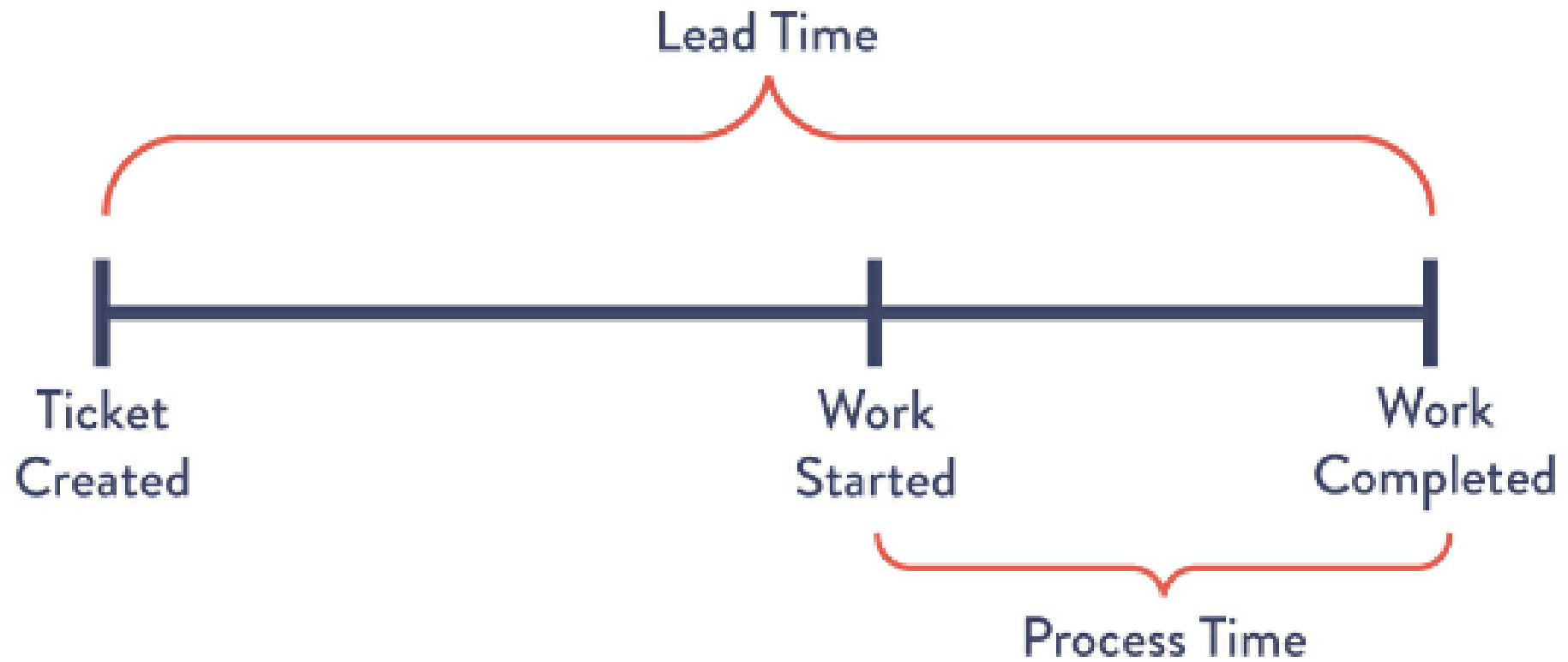
Testing, deployment and operation (Lean Manufacturing) – short and predictable

Goal

Testing, deployment, operations ||
design/development

- fast flow and high quality
- small batches
- build quality into every part of our value stream

Lead time vs Process Time



Long
deployment
lead time
(months)

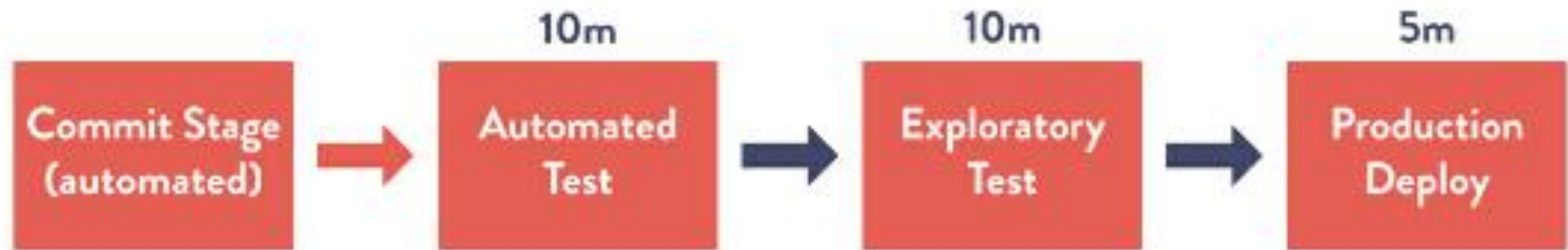
Nothing works at the end of the project when we merge all the Development team's changes together, resulting in code that no longer builds correctly or passes any of our tests. Fixing each problem requires days or weeks of investigation to determine who broke the code and how it can be fixed, and still results in poor customer outcomes



Short
deployment
lead time
(minutes)

developers receive fast, constant feedback on their work, enabling them to quickly and independently implement, integrate, and validate their code and have the code deployed into the production environment





➡ Automatic Approval
➡ Manual Approval

Total Cycle Time: 25m

Prerequisite

architecture that is modular, well encapsulated, and loosely coupled so that small teams can work with high degrees of autonomy, with failures being small and contained, and without causing global disruptions



Quality

Percentage of work complete and accurate
– no rework

DevOps Key principles

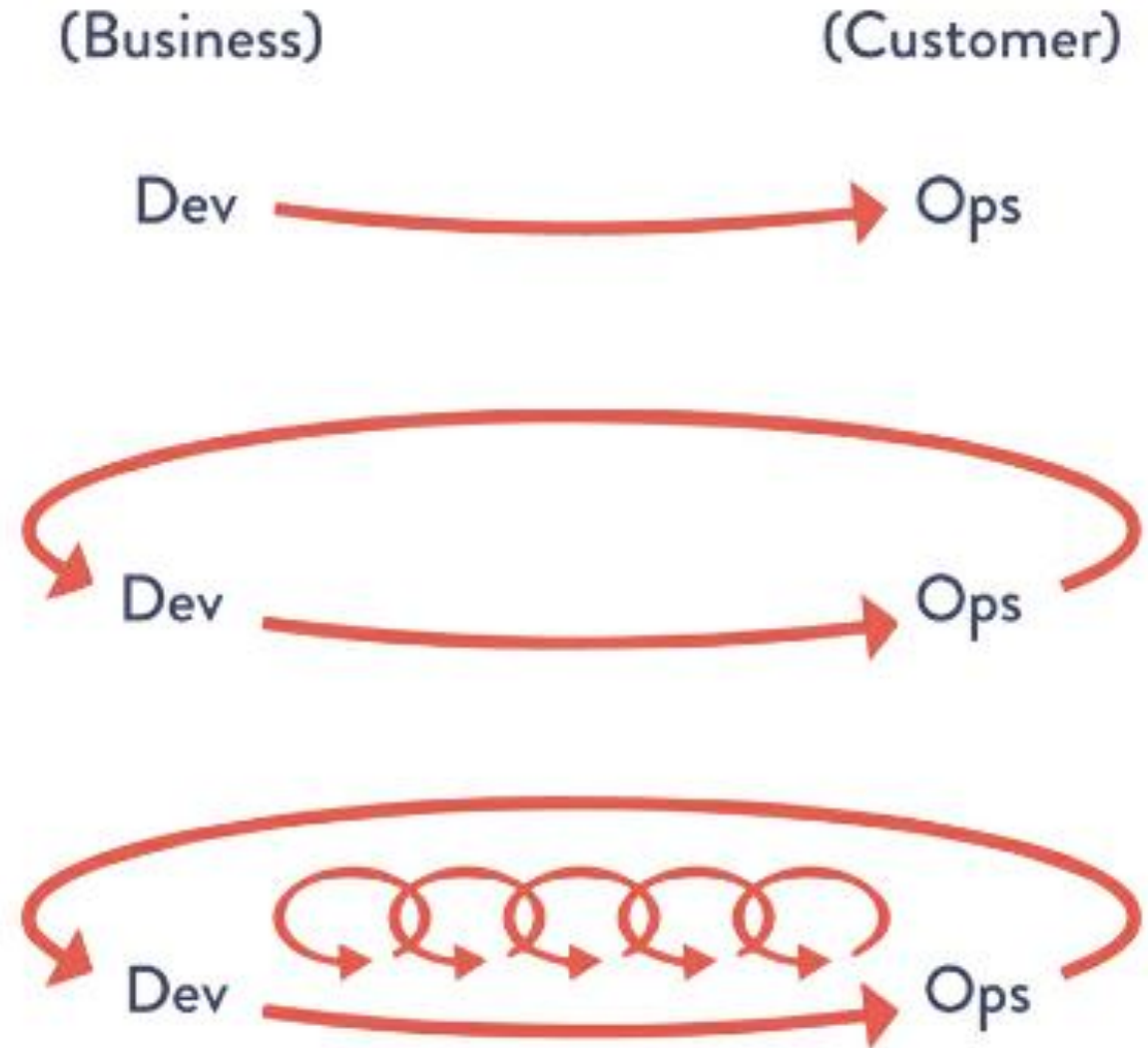
Principle of Flow

Principle of Feedback

Principle of Continual Learning and Experimentation



The three ways: The principles underpinning DevOps



The First Way

The Principles of Flow

The First Way



enables fast left-to-right flow of work from Development to Operations to the customer



to maximize flow

make work visible

reduce our batch sizes and intervals of work

build in quality by preventing defects from being passed to downstream work centers

constantly optimize for global goals

Objectives



decrease the amount of time required for
changes to be deployed into production



increase the reliability and quality of
those services

Make work visible



Use of visual work boards



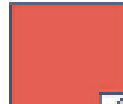
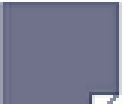
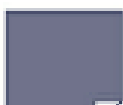
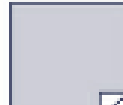
Use electronic cards to represent work items



Work is done when application is running successfully in production

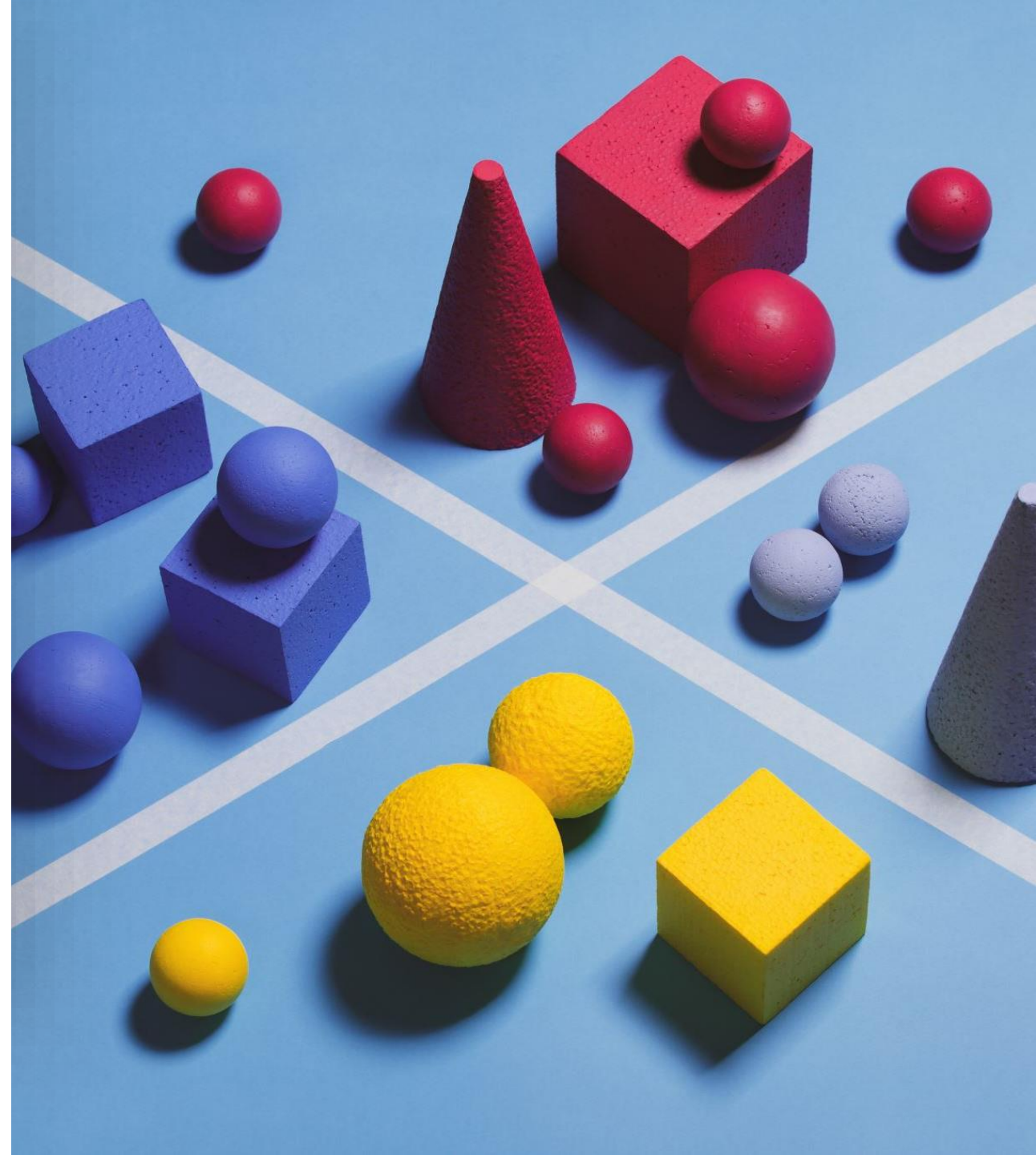


Single-task on the highest priority work until it is completed

		Development		Ops			
Ready	Investigate	Doing	Done	Doing	Done	UAT	Delivered
Expedite		→		→		→	
 						 	
 	 				 	 	 
		 				 	

Limit Work in Progress

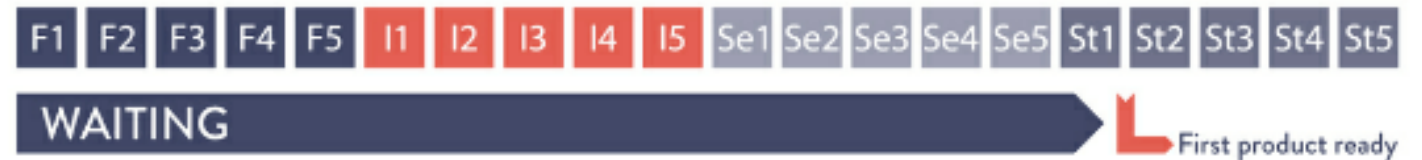
- Studies have shown that the time to complete even simple tasks, such as sorting geometric shapes, significantly degrades when multitasking
- Work in the technology value stream is far more cognitively complex than sorting geometric shapes, the effects of multitasking on process time is much worse.
- **limit multitasking**



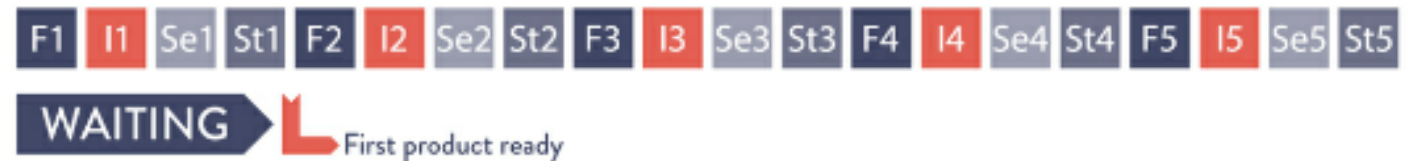
Reduce Batch Sizes

- key lessons in Lean – that in order to shrink lead times and increase quality, we must strive to continually **shrink batch sizes**
- get to market sooner
- Less risky in terms of defects

Large Batches

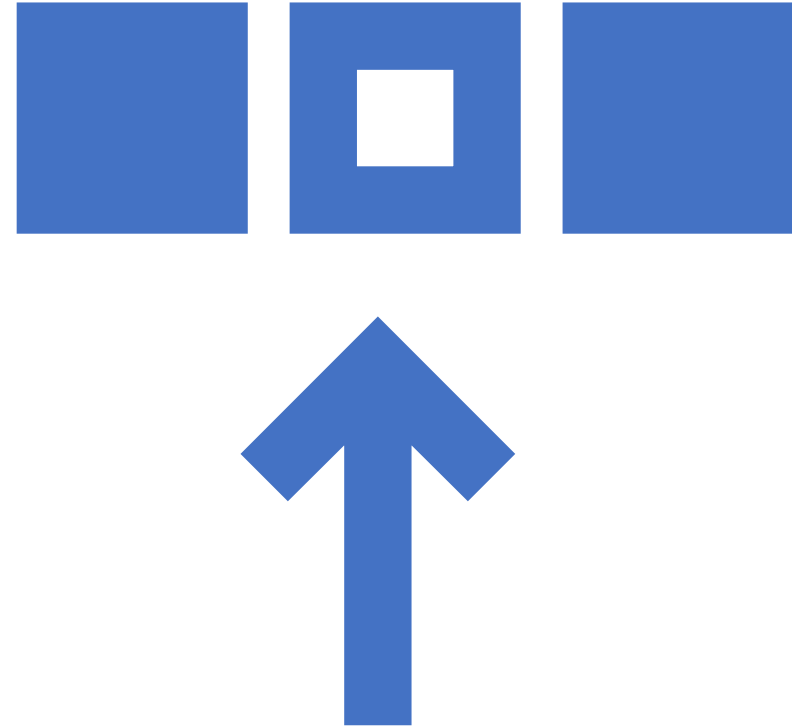


Single-Piece Flow



Single piece flow in the technology value stream

- continuous deployment – each change committed to version control is integrated, tested, and deployed into production.



Reduce number of handoffs



Each time the work passes from team to team, we require all sorts of communication



Potential queue where work will wait



Knowledge is lost

How to avoid handoffs?

Automating	automating significant portions of the work
Self-serve	building platforms and reorganizing teams so they can self-service builds, testing, and deployments to deliver value to the customer themselves instead of having to be constantly dependent on others

Continually Identify and Elevate Our Constraints



Environment creation



Code deployment



Test setup and run



Overly tight architecture

Eliminate Hardships and Waste in the Value Stream

Manufacturing

- inventory
- overproduction
- extra processing
- transportation
- waiting
- motion
- defects

Technology

- Partially done work
- Extra processes
- Extra features
- Task switching
- Waiting
- Motion
- Defects
- Non-standard / Manual work
- Heroics

The First Way - Results



continuous build, integration, test, and deployment processes



creating environments on demand



limiting work in process (WIP)



building systems and organizations that are safe to change

The Second Way

The Principles of Feedback

The Second Way

- enables the fast and constant flow of feedback from right to left at all stages of our value stream
- Requires
 - amplify feedback to prevent problems from happening again
 - enable faster detection and recovery.

Avoid

discover problems only when large failures are underway, such as a massive production outage or a security breach resulting in the theft of customer data



Goal

detect and remediate problems
while they are smaller, cheaper,
and easier to fix and create
organizational learning that we
integrate into future work

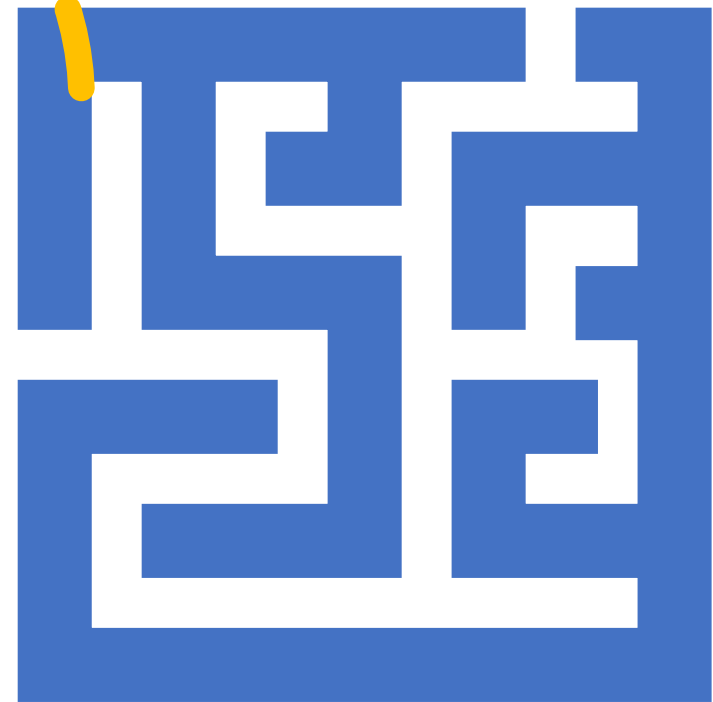


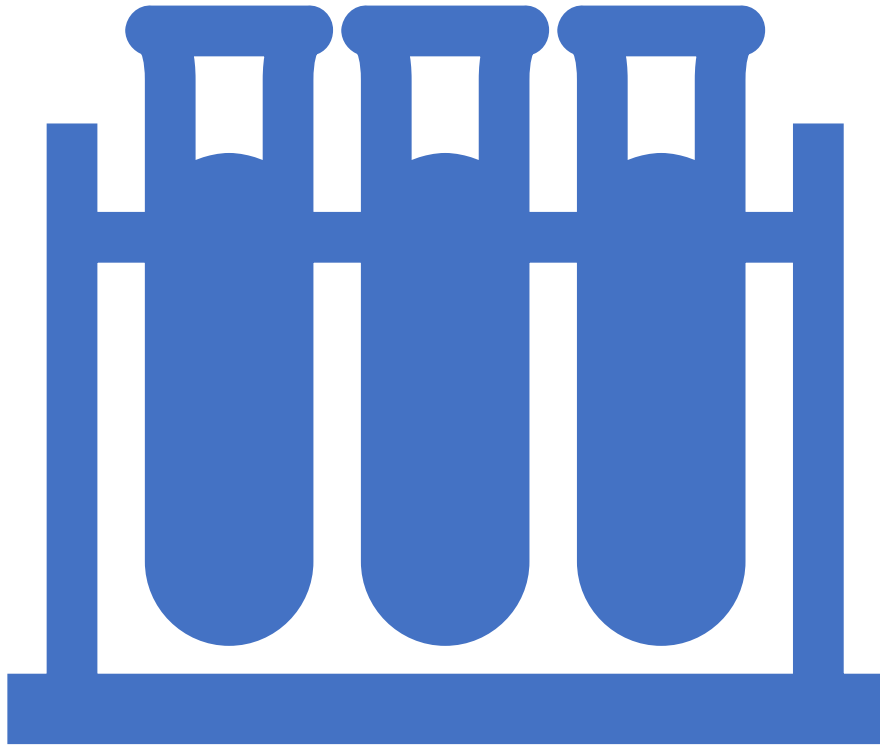
Working Safely within Complex Systems

- failure is inherent and inevitable in complex systems
- we must design a safe system of work, where we can perform work without fear, confident that most errors will be detected quickly, long before they cause catastrophic outcomes

Key aspects for working safely within Complex Systems

- Complex work is managed so that problems in design and operations are revealed
- Problems are swarmed and solved, resulting in quick construction of new knowledge
- New local knowledge is exploited globally throughout the organization
- Leaders create other leaders who continually grow these types of capabilities





See problems as they occur

- constantly test design and operating assumptions
- increase information flow in the system from as many areas as possible
- invalidate assumptions fast to increase resilience, agility, and ability to learn and innovate
- pervasive telemetry

Types of feedback in software development



Dev Tests: As a programmer, did I write the code I intended to write?



Continuous Integration (CI) and Testing: As a programmer, did I write the code I intended to write without violating any existing expectations in the code?



Exploratory Testing: Did we introduce any unintended consequences?



Acceptance Testing: Did I get the feature I asked for?



Stakeholder Feedback: As a team, are we headed in the right direction?



User Feedback: Are we producing something our customers/users love?

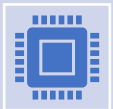
Push quality closer to the source



everyone in the value stream should find and fix problems in their area of control as part of their daily work



peer reviews of our proposed changes



automate as much of the quality checking typically performed by a QA or Information Security department as possible



tests can be performed on demand, enabling developers to quickly test their own code and even deploy those changes into production themselves

Enable Optimizing for Downstream Work Centers



Customers

External
Internal



Lean – most important customer is our next step downstream



Technology value stream – designing for operations, where operational non-functional requirements are prioritized as highly as user features

The Second Way - Results



create quality at the source and generate or embed knowledge where it is needed



create ever-safer systems of work where problems are found and fixed long before a catastrophic failure occurs



maximizes the opportunities for the organization to learn and improve

The Third Way

The Principles of Continual
Learning and Experimentation



The Third Way

Enables the creation of a generative, high-trust culture that supports a dynamic, disciplined, and scientific approach to experimentation and risk-taking, facilitating the creation of organizational learning, both from our successes and failures

Key objectives



reserve time for the improvement of daily work and time to further accelerate and ensure learning



consistently introduce stress into our systems to force continual improvement



simulate and inject failures in our production services under controlled conditions to increase our resilience



enable teams to rapidly and automatically adapt to an ever-changing environment

=> win in the marketplace

Types of culture (Dr. Westrum)



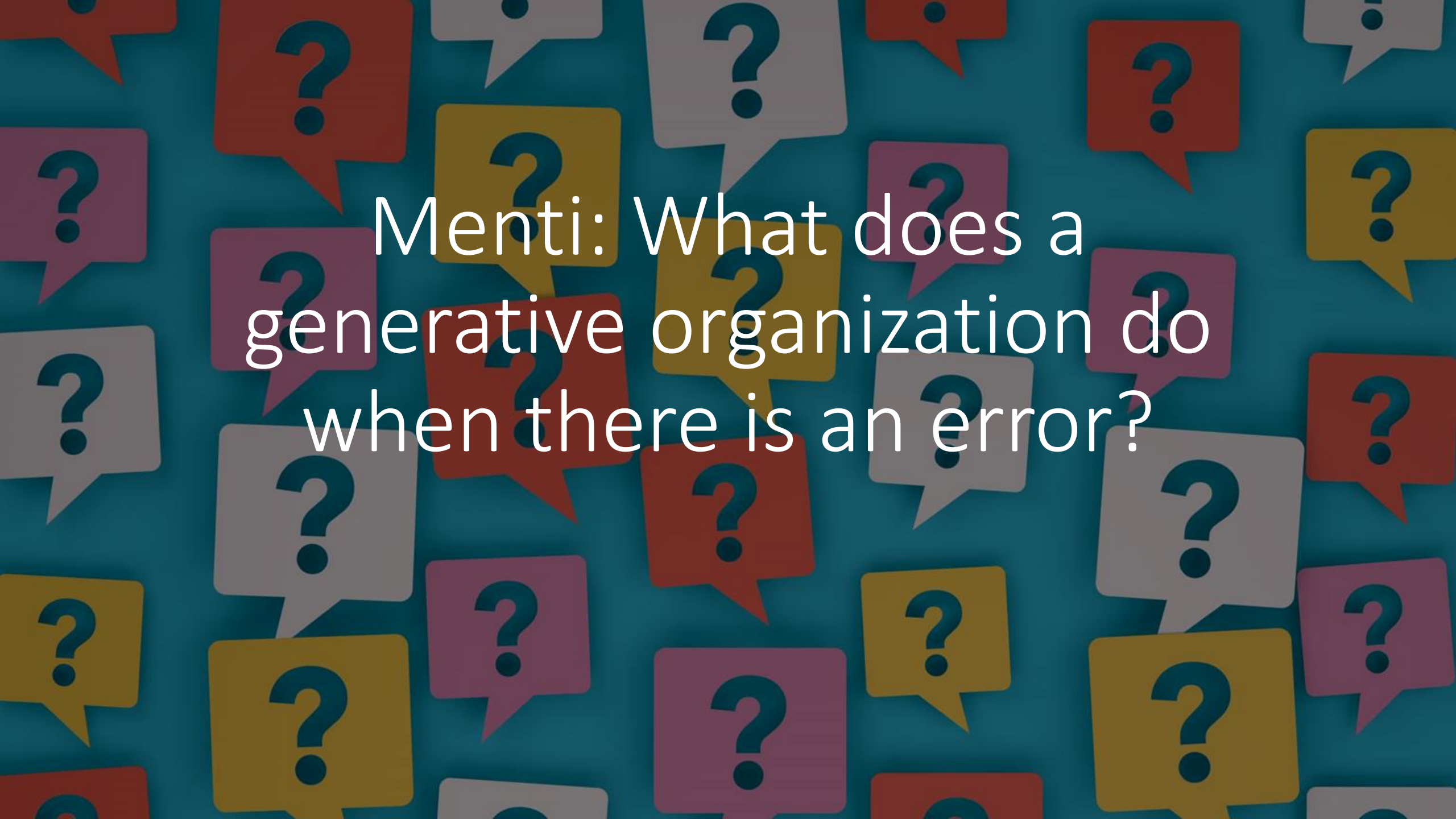
Pathological organizations – large amounts of fear and threat. People often hoard information, withhold it for political reasons, or distort it to make themselves look better. Failure is often hidden.



Bureaucratic organizations – rules and processes. Failure is processed through a system of judgment, resulting in either punishment or justice and mercy.



Generative organizations – actively seeking and sharing information to better enable the organization to achieve its mission. Responsibilities are shared throughout the value stream, and failure results in reflection and genuine inquiry.

The background of the slide is a dark teal color, densely populated with numerous speech bubbles of various colors including red, yellow, purple, and grey. Each speech bubble contains a large, dark blue question mark, creating a pattern that suggests a state of inquiry or confusion.

Menti: What does a
generative organization do
when there is an error?

Generative organizations – technology stream

when accidents and failures occur, instead of looking for human error, we look for how we can **redesign the system to prevent the accident** from happening again

Improvement of daily work



We improve daily work by explicitly reserving time to pay down technical debt, fix defects, and refactor and improve problematic areas of our code and environments



fix problems when it is not only easier and cheaper but also when the consequences are smaller

Transform Local Discoveries into Global Improvements



make all blameless post-mortem reports searchable by teams trying to solve similar problems



create shared source code repositories that span the entire organization, where shared code, libraries, and configurations that embody the best collective knowledge of the entire organization can be easily utilized

Inject Resilience Patterns into Our Daily Work

Manufacturing

- improve daily operations
- continually introduce tension to elevate performance
- engineer more resilience into the system

Technology stream

- always reduce deployment lead times
- increase test coverage
- decrease test execution times
- rearchitecting if necessary to increase developer productivity or increase reliability
- rehearse large scale failures
- may inject ever-larger scale faults into the production environment

“making all the right
decisions”

OR?

create the conditions so
their team can discover
greatness in their daily
work

Leaders Reinforce a
Learning Culture

In technology value stream



internal improvement processes – scientific approach and iterative method



perform experiments to ensure that the products we build actually help our internal and external customers achieve their goals

The Third Way - Results



learn faster than our competition and win in the marketplace



multiply the effects of new knowledge, transforming local discoveries into global improvements